



SOLIDProof

Bring trust into your projects

**Blockchain Security | Smart Contract Audits | KYC
Development | Marketing**

MADE IN GERMANY



Table of Contents

Heart Value Ecosystem (HeartLive) - Smart Contract Security Audit Report.....	4
Disclaimer.....	4
Executive Summary.....	5
Engagement Overview.....	5
Overall Security Rating.....	5
Findings Summary.....	5
Key Observations.....	6
Scope of Work.....	6
Target Description.....	6
Repository & Versioning.....	6
Reproducibility Metadata.....	7
Files & Components in Scope.....	7
In Scope.....	7
Out of Scope.....	7
Methodology & Tools.....	8
Approach Overview.....	8
Techniques Applied.....	8
Tools Used.....	8
Review Process.....	8
Severity & Status Definitions.....	9
House Standard Risk Model.....	9
Status Lifecycle.....	9
Minimum Evidence Requirements Per Finding.....	10
Logical Understanding / System Architecture.....	11
System Overview.....	11
Architecture Diagram.....	12
User Flows & Interaction Model.....	12
Trust Boundaries.....	13
External Dependencies & Integrations.....	13
Codebase Analysis Summary & Metrics.....	14
Code Overview.....	14
Dependency Health.....	14
Configuration & Secrets Management.....	14
Findings Overview.....	15
Summary Table.....	15
Detailed Findings.....	17
F-001: Insolvency due to missing token transfer in consensus building.....	17
F-002: Unlimited withdrawals in promotion vesting mechanism.....	19
F-003: Unsafe on-chain oracle via spot reserves.....	21
F-004: Cross-chain signature replay.....	23
F-005: Ineffective slippage protection.....	25
F-006: Cross-chain signature replay.....	27
F-007: Global variable causes unpredictable vesting rates.....	29
F-008: Centralization risk via excessive administrative privileges.....	30
F-009: Missing zero or dead address check.....	31
F-010: Missing event emission for point-type configuration.....	32
F-011: Uninitialized state variable (HTL_USDT_PAIR).....	33
F-012: Missing event emissions in critical functions.....	34
F-013: Missing event emissions.....	35
F-014: Loss of reward precision due to time rounding.....	36
F-015: Spot price fallback for unregistered tokens.....	37
F-016: Bootstrap window uses spot price.....	38

F-017: Reentrancy guard on transfer and transferFrom.....	39
F-018: State update after external call in promotion handling.....	40
F-019: Unchecked ERC20 transfer return value.....	41
F-020: Mock USDT address active in source.....	42
F-021: Potential precision loss in point-to-HTL conversion.....	43
F-022: Unchecked ERC20 transferFrom return value.....	44
F-023: Centralized oracle configuration.....	45
F-024: Testnet factory address active in source.....	46
F-025: Deployment dependency order.....	47
F-026: TVL manipulation via flash loan or donation.....	48
F-027: Reward inflation via price manipulation.....	49
F-028: Incorrect array size in vesting details view function.....	50
F-029: Unchecked ERC20 transfer return value.....	51
F-030: Gas cost from multiple transfer events per transaction.....	52
F-031: Gas-limit denial of service via unbounded array iteration.....	53
Centralisation, Access Control & Privilege Analysis.....	54
Overview.....	54
On-Chain Roles & Permissions.....	54
Centralisation Risk Summary.....	55
Conclusion.....	56
Summary.....	56
Positive Observations.....	56
Critical Action Items.....	56
Glossary.....	57

Heart Value Ecosystem (HeartLive) - Smart Contract Security Audit Report

Field	Value
Report Title	Smart Contract Security Audit - Heart Value Ecosystem (HeartLive)
Audit Type	Smart Contract (Solidity)
Version	v1.0
Date	2026-07-28
Classification	Public
Prepared by	SolidProof
Prepared for	Heart Value Ecosystem

Disclaimer

This report is provided on an "as-is" basis and reflects the state of the smart contract codebase at the time of the audit. It does not constitute legal, financial, or investment advice.

Limitations:

- Security audits are time-boxed engagements and cannot guarantee the absence of all vulnerabilities.
- This audit does not replace other security measures such as bug bounty programs, monitoring, incident response planning, or ongoing security reviews.
- Findings are based on the code, configurations, and documentation provided by the client at the time of engagement. Changes made after the audit may introduce new risks.
- The assessment does not cover economic, game-theoretic, tokenomics, or market manipulation attack vectors unless explicitly stated in the scope.
- Third-party dependencies and integrations (for example OpenZeppelin, PancakeSwap) are assumed to function as documented unless explicitly included in the audit scope.
- This report is owned by SolidProof and prepared exclusively for Heart Value Ecosystem. Distribution to third parties requires written consent.

Liability:

SolidProof assumes no liability for any losses, damages, or exploits arising from the use of the audited software, whether or not the vulnerabilities were identified in this report.

Executive Summary

Engagement Overview

SolidProof was engaged by Heart Value Ecosystem to perform a security audit of the HeartLive smart contract system, a set of Solidity contracts deployed on the BNB Smart Chain (BSC). The system implements a fee-on-transfer utility token (HTL), a central parameter and role registry, a time-weighted average price (TWAP) oracle, a multi-bucket vesting and reward engine, and a set of manager contracts that handle token purchases, consensus staking, loyalty-point exchange, and reward or task claims.

The engagement consisted of an initial review round, in which the findings documented in this report were identified, followed by a remediation verification round in which the client's fixes were independently validated against the original root cause. Findings that were confirmed fixed are marked **Verified**; findings that remain outstanding are marked **Open**.

Overall Security Rating

Rating	Description
Critical	Immediate action required - system is at risk of exploitation
Needs Improvement	Significant issues identified - remediation required before deployment
Moderate	Some issues found - manageable with timely fixes
Secure	No critical or high issues - minor improvements recommended
Excellent	Robust security posture - best practices observed

This engagement's rating: Secure (post-remediation)

All Critical and High severity findings identified during the initial review were remediated and independently verified. The residual risk is concentrated in centralisation of privileged roles and in a small number of pre-mainnet deployment configuration items (see Critical Action Items), none of which are open Critical or High severity issues.

Findings Summary

Severity	Count	Open	In Progress	Verified	Accepted Risk
Critical	0	0	0	0	0
High	3	0	0	3	0
Medium	5	1	0	4	0
Low	8	2	0	6	0
Informational	13	9	0	4	0
Gas Optimization	2	1	0	1	0
Total	31	13	0	18	0

Key Observations

- The most severe issues from the first round - a vesting insolvency, an unbounded promotion withdrawal, and single-block price manipulation via spot reserves - were all fixed and verified. Pricing across the system now flows through a dedicated TWAP oracle read via a cached view.
- Backend-signed authorization in the purchase and exchange flows was hardened against cross-chain and cross-instance signature replay by binding `block.chainid` and the contract address into every signed message.
- The main residual concern is centralisation: privileged roles take effect immediately without an on-chain timelock, and the backend signer is a trusted off-chain key. The intended mitigation is to hold these roles in a multi-signature wallet.
- Two deployment configuration constants (a mock USDT token and a testnet PancakeSwap factory) are still active in the source and must be switched to their production values before mainnet deployment.

Scope of Work

Target Description

The HeartLive system is a token-centric DeFi and loyalty ecosystem on the BNB Smart Chain. Users purchase the HTL token with a stablecoin (USDT) through the ConsensusManager, which swaps on PancakeSwap and opens time-locked vesting schedules with referral rewards. The VestingManager holds all locked balances and computes time-based releases and ecological rewards priced through a TWAP oracle. Loyalty points can be converted to and from HTL through the ExchangeManager under backend-signed authorization, and users read and claim rewards, vesting, and task allocations through the ExternalManager. All economic parameters, system wallets, and the backend signer address are stored centrally in the DAOParameters registry.

This engagement covered the on-chain smart contracts only. There was no backend, frontend, or infrastructure component in scope, and tokenomics and economic modeling were explicitly excluded.

Repository & Versioning

Field	Value
Repository	Provided as source snapshot (heart-ecosystem)
Commit Hash	Not committed at time of review - working-tree snapshot (see note)
Branch	main
Language/Compiler	Solidity 0.8.20
Frameworks	Foundry (forge)
Chain/Network	BNB Smart Chain (BSC), PancakeSwap V2 pairs

Note: The code was reviewed from a provided working-tree snapshot that did not contain a Git commit at the time of the engagement. For a reproducible final record, the client should tag the reviewed revision and record its commit hash here.

Reproducibility Metadata

Field	Value
Build/Test Environment	Foundry (forge); Solidity 0.8.20; via_ir = true; optimizer = true; optimizer_runs = 200
Tooling Version Lock	foundry.lock present; OpenZeppelin Contracts 4.9.6 (git submodule); forge-std (git submodule)
Test Command Set	forge build, forge test
Analysis Time Window	Initial review round: Q1 2026 (approx. 2026-01 to 2026-02); remediation verification completed 2026-06-12
Constraints and Assumptions	The in-system HTL token reverts on failed transfers; privileged roles are intended to be held by a multi-signature wallet; all priced tokens are registered in the TWAP oracle

Files & Components in Scope

#	File / Component	Layer	LoC
1	src/HeartToken.sol	Smart Contract	337
2	src/DAOParameters.sol	Smart Contract	216
3	src/MultiTokenTWAP.sol	Smart Contract	221
4	src/VestingManager.sol	Smart Contract	583
5	src/ConsensusManager.sol	Smart Contract	336
6	src/ExchangeManager.sol	Smart Contract	291
7	src/ExternalManager.sol	Smart Contract	148
	Total		2,132

In Scope

- The seven Solidity contracts listed above, including their access-control model, value-moving functions, pricing paths, signature verification, and vesting and reward logic.

Out of Scope

- Backend services, off-chain signer infrastructure, key management systems, and frontend or dApp code.
- Economic, tokenomics, and game-theoretic modeling.
- Third-party contracts and libraries (OpenZeppelin, PancakeSwap router/pair/factory), assumed correct unless explicitly stated.
- Social engineering and phishing vectors.
- Deployment scripts and CI/CD configuration.

Methodology & Tools

Approach Overview

The audit followed a multi-layered approach combining automated analysis, manual expert review, and adversarial reasoning against the smart contract layer. Because the entire system depends on a shared parameter registry and a shared price oracle, particular attention was paid to cross-contract trust relationships, pricing manipulation, and privileged-role reach.

Techniques Applied

Technique	Applied	Layer
Manual Code Review (line-by-line)	[x]	Smart Contract
Static Analysis (SAST)	[x]	Smart Contract
Symbolic / Automated Vulnerability Scan	[x]	Smart Contract
Fuzzing / Property-Based Testing	[x]	Smart Contract
Threat Modeling	[x]	Smart Contract
Gas / Performance Optimization Review	[x]	Smart Contract
Dynamic Analysis (DAST)	[]	Not in scope
API Security Testing (OWASP)	[]	Not in scope

Tools Used

Tool	Purpose	Version
Slither	Static analysis (Solidity)	0.11.5
Mythril	Symbolic execution	V0.24.8
Foundry	Testing and fuzzing framework	1.5.1-stable
Semgrep	SAST (multi-language)	1.159.0
Manual	Expert line-by-line review	n/a

Review Process

1. **Kick-off and Documentation Review** - Establish architecture understanding, trust assumptions, and threat model scope.
2. **Automated Analysis Pass** - Execute the agreed toolchain and preserve raw outputs for traceability.
3. **Manual Expert Review** - Perform line-by-line review and adversarial reasoning against critical flows (pricing, vesting release, signature verification).
4. **Exploitability Validation** - Validate practical exploit paths and business impact where feasible.
5. **Client Triage Session** - Confirm context, intended behavior, and remediation feasibility.
6. **Draft Reporting** - Document findings with technical evidence and client-facing risk statements.

7. **Remediation Verification** - Validate fixes against the original root cause and residual risk.
8. **Final Delivery** - Provide the report with a status lifecycle and a release-readiness statement.

Severity & Status Definitions

House Standard Risk Model

All findings use a unified house standard with mandatory mappings:

- **House Severity:** Critical / High / Medium / Low / Informational, plus Gas Optimization for non-security efficiency items.
- **CVSS v3.1 Base Score:** recorded for security-impacting findings.
- **CWE Mapping:** recorded for all findings where a matching CWE exists.

Severity is assigned using exploitability, impact, and business criticality. The CVSS base scores in this report are representative scores derived from the house severity assessment, because the source finding data did not include published CVSS vectors. House Severity is the primary client decision metric.

House Severity	CVSS Base Range	Typical Business Impact
Critical	9.0 - 10.0	Immediate risk to funds, data, or service integrity
High	7.0 - 8.9	Material risk requiring remediation before release
Medium	4.0 - 6.9	Contained but meaningful risk; remediation required on schedule
Low	0.1 - 3.9	Limited exploitability or low-impact weakness
Informational	0.0	Quality or hardening recommendation without direct security impact
Gas Optimization	0.0	Efficiency improvement without direct security impact

Status Lifecycle

Status	Definition	Required Evidence
Open	Finding confirmed and not remediated.	Reproduction steps and affected location
In Progress	Fix work has started but is not yet merged/released.	Linked work item/PR
Resolved	Client submitted a fix and claims closure.	Fix commit/PR reference
Verified	Auditor independently validated the fix.	Verification method and result
Accepted Risk	Client accepts residual risk with justification.	Formal risk acceptance statement and approver

Minimum Evidence Requirements Per Finding

Each finding includes: affected component and location, preconditions and threat-actor assumptions, exploit path or abuse scenario, technical and business impact, remediation reference, and verification result with residual risk.



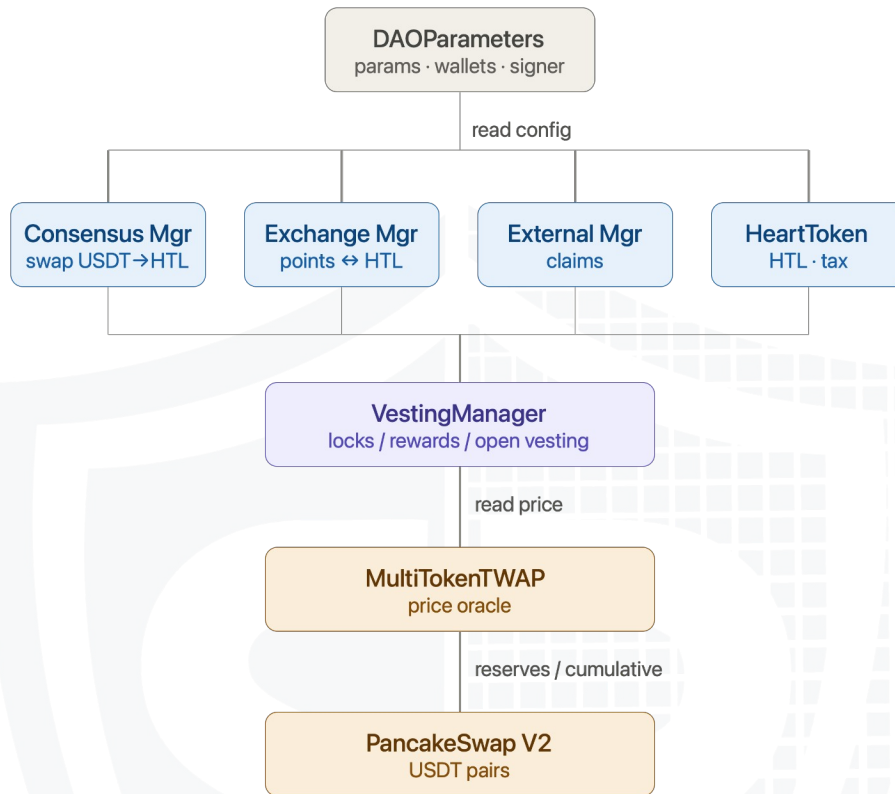
Logical Understanding / System Architecture

System Overview

The HeartLive system is composed of seven contracts that share a single configuration hub and a single price source:

- **HeartliveToken (HeartToken.sol)** - the HTL ERC20 token. Fixed total supply of 2,000,000,000 HTL minted once at deployment, a fee-on-transfer tax model capped at one percent, allocation pools, and role-based distribution. It cannot mint new supply.
- **DAOParameters (DAOParameters.sol)** - the central registry. It stores bounded economic parameters, system wallet addresses, point-type flags, and the backend signer address, guarded by a governance role (DAO_ROLE) and an admin role (ADMIN_ROLE).
- **MultiTokenTWAP (MultiTokenTWAP.sol)** - a time-weighted average price oracle for USDT trading pairs. It provides the tamper-resistant price used by the rest of the system.
- **VestingManager (VestingManager.sol)** - the multi-bucket vesting and reward engine. It holds locked balances per allocation type and releases them on time-based schedules, and computes ecological rewards using the TWAP price.
- **ConsensusManager (ConsensusManager.sol)** - the purchase and consensus-staking entry point. It swaps USDT for HTL on PancakeSwap, opens vesting schedules, and pays two-level referral rewards under backend-signed authorization.
- **ExchangeManager (ExchangeManager.sol)** - the loyalty-point exchange. It converts points to and from HTL under backend-signed authorization with a daily conversion limit.
- **ExternalManager (ExternalManager.sol)** - the front-facing gateway for reward, vesting, and task claims. It forwards to the VestingManager and collects the algorithm-service fee.

Architecture Diagram



User Flows & Interaction Model

- **Purchase / consensus staking:** A user submits a backend-signed voucher to the ConsensusManager, which swaps USDT for HTL (slippage bounded by the TWAP price), transfers the swapped HTL to the VestingManager, opens a vesting schedule, and credits referral rewards.
- **Reward and task claims:** A user calls the ExternalManager, which validates the vesting-type index and forwards to the VestingManager, where the payout is computed from recorded schedules and the cached TWAP price and then transferred.
- **Point exchange:** A user submits a backend-signed request to the ExchangeManager to convert points to HTL (or reverse), subject to a daily limit and a minimum-output check.

Trust Boundaries

Boundary	Trust Level	Notes
User -> Manager contracts	Untrusted	All user input and indexes validated on entry
Backend signer -> Consensus/Exchange	Trusted	Off-chain key; signatures bound to chain id and contract
Manager contracts -> VestingManager	Trusted	Role-gated (manager/external role), reentrancy-guarded
Contracts -> DAOParameters	Trusted	Central config; governance/admin controlled
Contracts -> MultiTokenTWAP	Trusted	Price source; governance controls pair registration
System -> PancakeSwap (router/pair/factory)	External	Liquidity and cumulative-price dependency

External Dependencies & Integrations

Dependency	Type	Risk Level	Notes
OpenZeppelin Contracts 4.9.6	Library	Low	Well-audited; version-pinned via submodule
PancakeSwap V2 (router/pair)	DEX / AMM	Medium	Swap execution and cumulative price feed
PancakeSwap Factory	DEX registry	Medium	Pair discovery for the spot path; testnet value in source
USDT (BEP-20)	Stablecoin	Medium	Payment and pricing base; mock value active in source
Backend Signer (off-chain)	Authorization	High	Key compromise enables unauthorized purchases/exchanges

Codebase Analysis Summary & Metrics

Code Overview

Metric	Value
Total Lines of Code (LoC)	2,132 (7 contracts in scope)
Language(s)	Solidity 0.8.20
Number of Contracts	7 in scope
Build System	Foundry (via_ir = true, optimizer_runs = 200)
External Imports	OpenZeppelin (ERC20, AccessControlEnumerable, ReentrancyGuard)
Test Coverage (%)	n/a

Dependency Health

Dependency	Version	Latest (branch)	Known CVEs	Pinned?	Notes
OpenZeppelin Contracts	4.9.6	4.9.x / 5.x	None known for used modules	Yes (submodule)	ReentrancyGuard, ERC20, AccessControlEnumerable
forge-std	submodule	-	n/a	Yes (submodule)	Test-only dependency

Configuration & Secrets Management

Item	Status	Notes
Hardcoded secrets in codebase	None found	No private keys or secrets in source
Deployment constants	Needs action	Mock USDT (DA0Parameters) and testnet factory (MultiTokenTWAP) active in source
Backend signer handling	Off-chain	Signer address stored on-chain; key managed off-chain (out of scope)
Production token address	Present but commented	Production BSC USDT 0x55d3...7955 is present but commented out

Findings Overview

Summary Table

ID	Title	Severity	Status	Contract
F-001	Insolvency due to missing token transfer in consensus building	High	Verified	VestingManager
F-002	Unlimited withdrawals in promotion vesting mechanism	High	Verified	VestingManager
F-003	Unsafe on-chain oracle via spot reserves	High	Verified	HeartToken
F-004	Cross-chain signature replay	Medium	Verified	ConsensusManager
F-005	Ineffective slippage protection	Medium	Verified	ConsensusManager
F-006	Cross-chain signature replay	Medium	Verified	ExchangeManager
F-007	Global variable causes unpredictable vesting rates	Medium	Verified	VestingManager
F-008	Centralization risk via excessive administrative privileges	Medium	Open	ExternalManager
F-009	Missing zero or dead address check	Low	Verified	ConsensusManager
F-010	Missing event emission for point-type configuration	Low	Verified	DAOParameters
F-011	Uninitialized state variable (HTL_USDT_PAIR)	Low	Verified	ExternalManager
F-012	Missing event emissions in critical functions	Low	Verified	ExternalManager
F-013	Missing event emissions	Low	Verified	VestingManager
F-014	Loss of reward precision due to time rounding	Low	Verified	VestingManager
F-015	Spot price fallback for unregistered tokens	Low	Open	MultiTokenTWAP
F-016	Bootstrap window uses spot price	Low	Open	MultiTokenTWAP
F-017	Reentrancy guard on transfer and transferFrom	Informational	Open	HeartToken
F-018	State update after external call in promotion handling	Informational	Open	ConsensusManager
F-019	Unchecked ERC20 transfer return value	Informational	Open	ConsensusManager
F-020	Mock USDT address active in source	Informational	Open	DAOParameters
F-021	Potential precision loss in point-to-HTL conversion	Informational	Verified	ExchangeManager
F-022	Unchecked ERC20 transferFrom return value	Informational	Open	ExchangeManager
F-023	Centralized oracle configuration	Informational	Open	MultiTokenTWAP

ID	Title	Severity	Status	Contract
F-024	Testnet factory address active in source	Informational	Open	MultiTokenTWAP
F-025	Deployment dependency order	Informational	Open	VestingManager
F-026	TVL manipulation via flash loan or donation	Informational	Verified	VestingManager
F-027	Reward inflation via price manipulation	Informational	Verified	VestingManager
F-028	Incorrect array size in vesting details view function	Informational	Verified	VestingManager
F-029	Unchecked ERC20 transfer return value	Informational	Open	VestingManager
F-030	Gas cost from multiple transfer events per transaction	Gas Optimization	Open	HeartToken
F-031	Gas-limit denial of service via unbounded array iteration	Gas Optimization	Verified	VestingManager

Detailed Findings

F-001: Insolvency due to missing token transfer in consensus building

Field	Value
ID	F-001
Severity	High
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:155-175
Category	Accounting / Business Logic
House Severity	High
CVSS v3.1	8.2 (representative)
CWE ID	CWE-840 (Business Logic Errors)
Exploit Preconditions	A user opens a consensus-building schedule through the purchase flow
Business Impact	Contract holds a liability larger than its balance; users cannot withdraw full balances
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

When creating a consensus-building schedule, the purchasing contract swapped the payment for tokens into its own balance but neither pushed those tokens to the vesting contract nor had them pulled, while the vesting contract recorded the full amount as owed. This created a liability larger than the balance actually held by the vesting contract.

Impact

The vesting contract became insolvent: the recorded obligations to users exceeded the tokens on hand. As a result, users could not withdraw their full balances, and the swapped tokens were effectively stuck in the purchasing contract.

Proof of Concept / Exploit Scenario

1. A user completes a consensus-building purchase; the ConsensusManager swaps USDT into HTL held in its own balance.
2. The VestingManager records the full swapped amount plus bonus as owed to the user.
3. No transfer of the swapped HTL to the VestingManager occurs, so the sum of recorded obligations exceeds the VestingManager balance.
4. Later withdrawals by users fail once the shortfall is reached.

Recommendation

Ensure the swapped tokens are actually delivered to the vesting contract before the schedule records them as part of the user balance.

Client Response

Addressed. The consensus contract now transfers the swapped tokens to the vesting contract during the purchase, and schedule creation only withdraws the bonus portion from the token pool before adding the already-delivered swapped amount to the recorded total.

Verification Method: Re-review of the purchase and schedule-creation path in the remediated snapshot. **Verification Result:** Pass - the vesting contract now holds exactly the bonus plus the swapped tokens it records as owed. **Residual Risk:** None.

F-002: Unlimited withdrawals in promotion vesting mechanism

Field	Value
ID	F-002
Severity	High
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:409-419, 501-516
Category	Access Control / Business Logic
House Severity	High
CVSS v3.1	8.6 (representative)
CWE ID	CWE-837 (Improper Enforcement of a Single, Unique Action)
Exploit Preconditions	A promotion vesting schedule exists and its start time is more than one month in the past
Business Impact	An entire multi-year allocation can be drained almost instantly
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The promotion release logic intended to release a small percentage once per month but previously only checked that the start time was more than a month in the past. Once that condition became true it stayed true forever, so the guard never prevented subsequent releases.

Impact

A user could call the claim repeatedly to release batch after batch, draining the entire multi-year promotion allocation almost instantly rather than over the intended schedule.

Proof of Concept / Exploit Scenario

1. Wait until a promotion schedule is older than one month.
2. Call the release function repeatedly within the same block or in quick succession.
3. Because only `startTime + 1 month < now` is checked and no last-claim timestamp is enforced, each call releases another batch until the allocation is exhausted.

Recommendation

Track the last claim time and require that a full interval has elapsed before allowing another release, updating the timestamp on each claim.

Client Response

Addressed. Each schedule now records a last-release timestamp. The release routine returns nothing until a full month has elapsed since that timestamp, the releasable

amount is computed from the number of whole months passed, and the timestamp is updated on every successful claim.

Verification Method: Re-review of the release routine and its timestamp bookkeeping. **Verification Result:** Pass - repeated calls in the same block or shortly after release nothing further. **Residual Risk:** None.



F-003: Unsafe on-chain oracle via spot reserves

Field	Value
ID	F-003
Severity	High
Status	Verified
Contract	HeartToken
Location	HeartToken.sol:13-33
Category	Price Oracle Manipulation
House Severity	High
CVSS v3.1	8.1 (representative)
CWE ID	CWE-345 (Insufficient Verification of Data Authenticity)
Exploit Preconditions	Attacker can execute a flash loan or large swap against the trading pair
Business Impact	Corrupted pricing inflates rewards and discounts in dependent contracts
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The token previously implemented a price oracle by reading instantaneous reserves directly from the trading pair, ignoring any time-weighting. This allowed an attacker to manipulate the reported price within a single transaction using a flash loan or large swap, which in turn corrupted dependent financial calculations such as rewards and discounts in other contracts.

Impact

A single-transaction price move could inflate ecological rewards or distort discounts and slippage checks that read this price, allowing value to be extracted from the system.

Proof of Concept / Exploit Scenario

1. Take a flash loan and swap heavily against the HTL/USDT pair to move the instantaneous reserves.
2. Within the same transaction, trigger a dependent operation (reward or discount calculation) that reads the spot price.
3. Benefit from the manipulated price, then unwind the swap and repay the flash loan.

Recommendation

Abandon spot reserves for price determination and rely on a tamper-resistant feed such as a time-weighted average price computed over a defined window.

Client Response

Addressed. The instantaneous spot-reserve price function has been removed from the token. Pricing is now provided by a dedicated TWAP oracle, and reward calculations

read the cached average price through a view function, so the reported price cannot be moved within a single transaction.

Verification Method: Confirmed removal of the spot-price function and that consumers read the cached TWAP view. **Verification Result:** Pass - single-transaction manipulation path removed. **Residual Risk:** One residual point is tracked separately (F-015): the price reader falls back to a spot reading for tokens not registered in the oracle, so every priced token must be registered.



F-004: Cross-chain signature replay

Field	Value
ID	F-004
Severity	Medium
Status	Verified
Contract	ConsensusManager
Location	ConsensusManager.sol:141-150
Category	Signature Verification / Replay
House Severity	Medium
CVSS v3.1	6.5 (representative)
CWE ID	CWE-294 (Authentication Bypass by Capture-replay)
Exploit Preconditions	Same backend signer used across multiple chains or deployments
Business Impact	A valid signature can be replayed to execute unauthorized purchases or vesting
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The signature verification mechanism reconstructed the signed message hash from the business parameters but failed to include `block.chainid` or the contract address. This made a signature valid on any blockchain where the system is deployed with the same backend signer.

Impact

An attacker could replay a valid signature from one chain on another chain (or another deployment) to execute unauthorized purchases or open vesting schedules.

Proof of Concept / Exploit Scenario

1. Capture a valid backend-signed voucher used on chain A.
2. Submit the same voucher to the same contract logic deployed on chain B, where the signer is identical.
3. Because the hash omits chain id and contract address, verification succeeds and the purchase executes again.

Recommendation

Update the message hash to include `block.chainid` and `address(this)`, binding the signature strictly to a specific contract instance on a specific chain.

Client Response

Addressed. The signed message is now built with `abi.encode(block.chainid, address(this), user, usdtAmount, lockPeriod, otherTAddress,`

`otherTAmount, voucherId, deadline`). The switch from `abi.encodePacked` to `abi.encode` also removes ambiguity from packed dynamic arguments.

Verification Method: Re-review of the message-hash construction. **Verification Result:** Pass - signatures are now bound to chain and contract instance. **Residual Risk:** None. All conversions still depend on a trusted backend signer whose key must be protected (see Centralisation analysis).



F-005: Ineffective slippage protection

Field	Value
ID	F-005
Severity	Medium
Status	Verified
Contract	ConsensusManager
Location	ConsensusManager.sol:120-202
Category	Price Oracle Manipulation / Slippage
House Severity	Medium
CVSS v3.1	6.1 (representative)
CWE ID	CWE-345 (Insufficient Verification of Data Authenticity)
Exploit Preconditions	Attacker can front-run the purchase transaction (sandwich)
Business Impact	Users receive materially less HTL than expected on a purchase
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The contract previously calculated the minimum acceptable output for the swap by querying the router for the current pool state (`getAmountsOut`) in the same transaction. Deriving the minimum from the live pool state renders slippage protection ineffective, because the same state can be manipulated within the block.

Impact

An attacker could front-run the trade to skew the reserves and force the contract to accept a much lower output, extracting value through a classic sandwich attack.

Proof of Concept / Exploit Scenario

1. Observe a pending purchase in the mempool.
2. Front-run it with a swap that skews the pair reserves.
3. The victim transaction computes its minimum output from the already-skewed live state, so the weakened check passes and the victim receives less HTL.
4. Back-run to restore reserves and capture the difference.

Recommendation

Do not derive the minimum output from the live pool state inside the same transaction. Use a time-weighted average price to establish a minimum expected exchange rate that cannot be manipulated within a single block.

Client Response

Addressed. The minimum output is now derived from a TWAP obtained from the dedicated oracle (`getTWAPPrice`), combined with a configurable tolerance (`SWAPPROTECT`, default fifteen percent) and a minimum price floor.

Verification Method: Re-review of the minimum-output derivation. **Verification Result:** Pass - the reference price now comes from a TWAP rather than the instantaneous reading. **Residual Risk:** None; the tolerance parameter should be kept conservative.



F-006: Cross-chain signature replay

Field	Value
ID	F-006
Severity	Medium
Status	Verified
Contract	ExchangeManager
Location	ExchangeManager.sol:110-113, 165, 185, 209-211
Category	Signature Verification / Replay
House Severity	Medium
CVSS v3.1	6.5 (representative)
CWE ID	CWE-294 (Authentication Bypass by Capture-replay)
Exploit Preconditions	Same backend signer used across multiple chains or deployments
Business Impact	Points could be double-spent or rewards claimed on multiple networks
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The contract uses signatures to authorize point exchanges and payouts. The message hashes previously omitted `block.chainid` and the contract address, so a signature produced for one chain could be replayed on another chain or deployment using the same backend signer.

Impact

An attacker could replay a valid conversion or payout signature across networks, potentially double-spending points or claiming rewards on multiple deployments.

Proof of Concept / Exploit Scenario

1. Capture a valid signed exchange request from chain A.
2. Replay it against the equivalent deployment on chain B with the same signer.
3. Verification succeeds because the hash is not bound to chain or contract, so the exchange executes again.

Recommendation

Include `block.chainid` and `address(this)` in every signed message so each signature is bound to a specific contract instance on a specific chain.

Client Response

Addressed. Every signature-gated function now builds its message hash with `abi.encode` starting with `block.chainid` and `address(this)` before the business parameters. The replay-protection bookkeeping in the shared verifier remains in place.

Verification Method: Re-review of all signature-gated entry points. **Verification Result:** Pass - each signature is bound to chain and contract instance; per-user replay buffer retained. **Residual Risk:** None. Conversions still depend on the trusted backend signer.



F-007: Global variable causes unpredictable vesting rates

Field	Value
ID	F-007
Severity	Medium
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:501-516
Category	Business Logic / Race Condition
House Severity	Medium
CVSS v3.1	5.3 (representative)
CWE ID	CWE-362 (Concurrent Execution using Shared Resource)
Exploit Preconditions	Multiple users interacting with the promotion release in the same period
Business Impact	Non-deterministic release rates; users can game timing to maximize releases
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The contract previously used a single global value, updated on every interaction, to compute the release rate for all users. One user's deposit or withdrawal directly altered the next user's release rate.

Impact

The schedule was non-deterministic and exploitable: users were incentivized to time their actions around others to maximize their own release rate, and one user's actions changed the calculation for the next.

Recommendation

Do not derive the release rate from a single global value that any user can shift; use a deterministic per-schedule rate.

Client Response

Addressed. The release rate is no longer derived from a shared global value. The promotion release now uses a deterministic configured rate clamped between a minimum and a maximum, multiplied by the number of whole months passed for the individual schedule.

Verification Method: Re-review of the release-rate derivation. **Verification Result:** Pass - one user's actions no longer change the release calculation for another. **Residual Risk:** None.

F-008: Centralization risk via excessive administrative privileges

Field	Value
ID	F-008
Severity	Medium
Status	Open (partially mitigated)
Contract	ExternalManager
Location	ExternalManager.sol:123-148
Category	Centralisation / Access Control
House Severity	Medium
CVSS v3.1	6.5 (representative)
CWE ID	CWE-269 (Improper Privilege Management)
Exploit Preconditions	Compromise or misuse of the privileged role key
Business Impact	Configuration or wallet changes apply instantly with no reaction window
Remediation Reference	Partially mitigated; on-chain timelock not implemented

Description

The privileged role can modify critical infrastructure and trigger fund movements with immediate effect. Without an on-chain timelock, configuration or wallet changes apply instantly, leaving users no window to react in the event of a malicious update or a key compromise.

Impact

If the privileged key is compromised or misused, configuration and wallet changes take effect immediately. The risk is reduced, but not removed, when the role is held by a multi-signature wallet rather than a single externally owned account.

Recommendation

Hold the privileged role in a multi-signature wallet and route critical configuration changes through a timelock so the community can observe and react before changes take effect.

Client Response

Partially mitigated. Fee-wallet management moved into the parameters contract, where setters reject zero and no-op addresses and emit events, and the wallet values here are synchronized from there rather than set arbitrarily. However, the privileged role still takes effect immediately without an on-chain timelock.

Verification Method: Re-review of the wallet-management path and role behavior. **Verification Result:** Partial - input hygiene and event coverage improved; immediate-effect privilege remains. **Residual Risk:** Open. Depends on the role being held by a multi-signature wallet (the team's stated intent); an on-chain timelock is recommended for defence in depth.

F-009: Missing zero or dead address check

Field	Value
ID	F-009
Severity	Low
Status	Verified
Contract	ConsensusManager
Location	ConsensusManager.sol:295-336
Category	Input Validation
House Severity	Low
CVSS v3.1	3.7 (representative)
CWE ID	CWE-20 (Improper Input Validation)
Exploit Preconditions	Privileged setter called with a zero or dead address
Business Impact	Accidental zero value could silently break price feeds or admin control
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

Addresses passed to privileged setters could be set to the zero or dead address, since inputs were not validated.

Impact

An accidental zero value would silently break price feeds or admin control.

Recommendation

Check that addresses cannot be set to the zero or dead address.

Client Response

Addressed. `setAdminManager` reverts on the zero address, `setHTLOracle` rejects a zero oracle, and the constructor checks the token, DAO and vesting addresses. The remaining open input is the referrer in `SetReferrer`, where a zero value simply means no referrer and carries no risk.

Verification Result: Pass. **Residual Risk:** None.

F-010: Missing event emission for point-type configuration

Field	Value
ID	F-010
Severity	Low
Status	Verified
Contract	DAOParameters
Location	DAOParameters.sol:169-174
Category	Logging / Monitoring
House Severity	Low
CVSS v3.1	2.0 (representative)
CWE ID	CWE-778 (Insufficient Logging)
Exploit Preconditions	None (observability weakness)
Business Impact	Off-chain indexers cannot reliably track authorized point types
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The `setPointType` function approved or disabled point types without emitting an event, making it difficult for off-chain indexers to track which point types were authorized without polling state.

Recommendation

Define and emit an event inside `setPointType`.

Client Response

Addressed. A `PointTypeUpdated` event is now declared and emitted with the caller, the key and the new value. The function also rejects empty keys and no-op updates.

Verification Result: Pass. **Residual Risk:** None.

F-011: Uninitialized state variable (HTL_USDT_PAIR)

Field	Value
ID	F-011
Severity	Low
Status	Verified
Contract	ExternalManager
Location	ExternalManager.sol:40-61
Category	Initialization
House Severity	Low
CVSS v3.1	3.5 (representative)
CWE ID	CWE-665 (Improper Initialization)
Exploit Preconditions	Deployment without the manual post-deployment configuration call
Business Impact	Reward functions unusable until manually configured
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

A pair address variable used to fetch the token price was declared but not initialized in the constructor, creating a dependency on a manual post-deployment configuration call that, if overlooked, would render reward functions unusable.

Recommendation

Initialize the price source on deployment so reward functions are usable without manual post-deployment configuration.

Client Response

Addressed. The contract no longer holds its own price pair variable. Reward and vesting reads are delegated to the vesting contract, which sources prices from the dedicated TWAP oracle.

Verification Result: Pass. **Residual Risk:** None.

F-012: Missing event emissions in critical functions

Field	Value
ID	F-012
Severity	Low
Status	Verified
Contract	ExternalManager
Location	ExternalManager.sol:31-148
Category	Logging / Monitoring
House Severity	Low
CVSS v3.1	2.0 (representative)
CWE ID	CWE-778 (Insufficient Logging)
Exploit Preconditions	None (observability weakness)
Business Impact	Reward history, vesting status and configuration changes hard to track off-chain
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

Several functions executed without emitting events, including asset-moving claim functions and administrative configuration changes.

Recommendation

Emit events for asset-moving user actions and for administrative configuration changes.

Client Response

Addressed. The algorithm cost payment, the admin role changes and the DAO wallet synchronization now emit events. The reward, vesting and task claim flows run through the vesting contract, which emits dedicated claim events.

Verification Result: Pass. **Residual Risk:** None.

F-013: Missing event emissions

Field	Value
ID	F-013
Severity	Low
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:71-75
Category	Logging / Monitoring
House Severity	Low
CVSS v3.1	2.0 (representative)
CWE ID	CWE-778 (Insufficient Logging)
Exploit Preconditions	None (observability weakness)
Business Impact	Off-chain systems forced to poll state; hard to monitor in real time
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The contract emitted no events for critical state changes such as creating vesting schedules, releasing tokens or claiming rewards.

Recommendation

Emit specific events for vesting creation, token release and reward claims, with an indexed user address.

Client Response

Addressed. The contract now emits a vesting-created event, a vesting-claimed event and a reward-claimed event, with the user address indexed.

Verification Result: Pass. **Residual Risk:** None.

F-014: Loss of reward precision due to time rounding

Field	Value
ID	F-014
Severity	Low
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:202, 283-292
Category	Incorrect Calculation
House Severity	Low
CVSS v3.1	3.1 (representative)
CWE ID	CWE-682 (Incorrect Calculation)
Exploit Preconditions	Frequent reward claims
Business Impact	Accrued time silently erased; effective yield below advertised rate
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The reward duration was computed by integer division into whole days, but the checkpoint was then advanced to the exact current timestamp, discarding the partial-day remainder. Over frequent claims this rounding error accumulated.

Recommendation

Advance the reward checkpoint only by the time actually accounted for, so the unused partial day is preserved for the next calculation.

Client Response

Addressed. The reward checkpoint is now aligned to the start of the day and advanced to the start of the current day rather than the exact current second, so the partial-day remainder is preserved.

Verification Result: Pass. **Residual Risk:** None.

F-015: Spot price fallback for unregistered tokens

Field	Value
ID	F-015
Severity	Low
Status	Open
Contract	MultiTokenTWAP
Location	MultiTokenTWAP.sol:111-152
Category	Price Oracle Manipulation
House Severity	Low
CVSS v3.1	3.7 (representative)
CWE ID	CWE-345 (Insufficient Verification of Data Authenticity)
Exploit Preconditions	A priced token is used before being registered via addToken
Business Impact	Consumers may receive a manipulable spot price during purchases and claims
Remediation Reference	Open - operational control required

Description

The updating price reader falls back to an instantaneous spot price computed from current reserves when the requested token has not been registered through addToken. Consumers relying on this reader during purchases and claims would then receive a spot price, which can be moved within a single block, partially reverting the protection the TWAP is meant to provide. The cached view reader does revert for unregistered tokens, so reward accrual is unaffected, and the spot fallback enforces a minimum liquidity threshold, which limits but does not remove the manipulation surface.

Impact

Any priced token that is not registered can be priced from a manipulable spot reading, re-opening a reduced version of F-003 for that token.

Recommendation

Register every priced token through addToken before it is used, and consider reverting in the updating price reader when a token has not been registered instead of silently returning a spot reading.

Client Response

[Pending client response]

Verification Method: Not yet verified - open. **Verification Result:** Open. **Residual Risk:** Follow-up required. Every priced token must be registered for the average-price protection to be effective.

F-016: Bootstrap window uses spot price

Field	Value
ID	F-016
Severity	Low
Status	Open
Contract	MultiTokenTWAP
Location	MultiTokenTWAP.sol:122-131
Category	Price Oracle Manipulation
House Severity	Low
CVSS v3.1	3.1 (representative)
CWE ID	CWE-345 (Insufficient Verification of Data Authenticity)
Exploit Preconditions	Reads within the first averaging period after a token is registered
Business Impact	Reported price reflects spot rather than a true average during the initial window
Remediation Reference	Open - operational awareness required

Description

On the first update for a registered token the average price is seeded from the current spot price, and subsequent reads before a full period has elapsed return that seeded value. During this initial window the reported price is effectively a spot price and could be influenced around the moment of registration. After the first full period the value is replaced by a genuine cumulative-price average. The liquidity threshold on the spot path limits the impact.

Recommendation

Be aware that the first reading after a token is registered, and any reading before a full period has elapsed, reflects the spot price captured at initialization rather than a true average. Avoid price-sensitive operations during this bootstrap window.

Client Response

[Pending client response]

Verification Method: Not yet verified - open. **Verification Result:** Open. **Residual Risk:** Follow-up required; effect is bounded by the liquidity threshold and ends after the first full period.

F-017: Reentrancy guard on transfer and transferFrom

Field	Value
ID	F-017
Severity	Informational
Status	Open
Contract	HeartToken
Location	HeartToken.sol:118-137
Category	Integration / Design Note
House Severity	Informational
CVSS v3.1	0.0
CWE ID	N/A (design characteristic, not a defect)
Exploit Preconditions	An integrating contract moves HTL inside its own guarded section
Business Impact	Integrations may revert unexpectedly
Remediation Reference	N/A - documented behavior

Description

The token overrides `transfer` and `transferFrom` and marks them as reentrancy-guarded. This is a defensive choice given the fee-on-transfer logic, but it makes the token unusual: any external contract that itself holds the same guard and tries to move the token inside a guarded section could revert. No exploitable issue was identified.

Recommendation

Confirm that no integrating contract performs token transfers in the middle of its own guarded flow.

Verification Result: Open (documentation note). **Residual Risk:** Informational.

F-018: State update after external call in promotion handling

Field	Value
ID	F-018
Severity	Informational
Status	Open
Contract	ConsensusManager
Location	ConsensusManager.sol:226-239
Category	Checks-Effects-Interactions
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-691 (Insufficient Control Flow Management)
Exploit Preconditions	Reentrancy into the guarded entry point (mitigated)
Business Impact	Defence-in-depth gap; no exploit identified
Remediation Reference	Open - hardening recommendation

Description

In the promotion handling routine the referrer accounting is read and written around external calls into the vesting contract. The public entry point carries a reentrancy guard and the vesting contract is a trusted in-system contract that also guards reentrancy, so the practical risk is limited, but the ordering does not follow checks-effects-interactions.

Recommendation

Compute and persist the referrer accounting before the external vesting calls, or continue relying on the reentrancy guard and the trusted vesting contract.

Verification Result: Open. **Residual Risk:** Informational (mitigated by reentrancy guard and trusted callee).

F-019: Unchecked ERC20 transfer return value

Field	Value
ID	F-019
Severity	Informational
Status	Open
Contract	ConsensusManager
Location	ConsensusManager.sol:205
Category	Unchecked Return Value
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-252 (Unchecked Return Value)
Exploit Preconditions	A token that returns false instead of reverting
Business Impact	None with the in-system token, which reverts on failure
Remediation Reference	Open - hardening recommendation

Description

The transfer of swapped tokens to the vesting contract ignores the boolean return value of the token transfer. The in-system token reverts on a failed transfer rather than returning false, so funds cannot be silently lost here.

Recommendation

Use SafeERC20 safeTransfer, or continue relying on the fact that the in-system token reverts on failure.

Verification Result: Open. **Residual Risk:** Informational.

F-020: Mock USDT address active in source

Field	Value
ID	F-020
Severity	Informational (deployment-blocking)
Status	Open
Contract	DAOParameters
Location	DAOParameters.sol:30
Category	Deployment Configuration
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-547 (Use of Hard-coded, Security-relevant Constants)
Exploit Preconditions	Mainnet deployment with the mock token still configured
Business Impact	The entire pricing and payment flow would use a test token
Remediation Reference	Open - must be changed before mainnet deployment

Description

The USDT address constant is set to a mock token used for testing (0x82aFa609e0553c46FCd6361EC3840d33842019CC), while the production BSC address (0x55d398326f99059fF775485246999027B3197955) is left commented out. This is expected during testing but must be switched before mainnet deployment.

Recommendation

Before mainnet deployment, switch USDT_ADDRESS to the production token address that is currently commented out.

Verification Result: Open. **Residual Risk:** Deployment-blocking; must be resolved before mainnet.

F-021: Potential precision loss in point-to-HTL conversion

Field	Value
ID	F-021
Severity	Informational
Status	Verified
Contract	ExchangeManager
Location	ExchangeManager.sol:142-153
Category	Incorrect Calculation
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-682 (Incorrect Calculation)
Exploit Preconditions	Very small conversions where integer division rounds down to zero
Business Impact	A user could previously spend points and receive no tokens
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The formula converting points to tokens uses integer arithmetic. When the denominator is large relative to the numerator the result can truncate to zero, meaning a user could previously have spent points and received no tokens in return.

Recommendation

Revert conversions that would round down to zero tokens, and consider a higher-precision scaling factor before the final division.

Client Response

Addressed for the loss-of-funds aspect. The conversion now reverts with "HTL amount too small" when the division rounds to zero. The scaling factor remains modest, so very small conversions during high token prices still revert rather than return fractional value, which is acceptable and safe.

Verification Result: Pass (loss-of-funds path closed). **Residual Risk:** None.

F-022: Unchecked ERC20 transferFrom return value

Field	Value
ID	F-022
Severity	Informational
Status	Open
Contract	ExchangeManager
Location	ExchangeManager.sol:173
Category	Unchecked Return Value
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-252 (Unchecked Return Value)
Exploit Preconditions	A token that returns false instead of reverting
Business Impact	None with the in-system token, which reverts on failure
Remediation Reference	Open - hardening recommendation

Description

The points-to-HTL reversal moves tokens with `transferFrom` and ignores the boolean return value. The in-system token reverts on a failed transfer, so value cannot be silently lost.

Recommendation

Use SafeERC20 `safeTransferFrom`, or continue relying on the in-system token reverting on failure.

Verification Result: Open. **Residual Risk:** Informational.

F-023: Centralized oracle configuration

Field	Value
ID	F-023
Severity	Informational
Status	Open
Contract	MultiTokenTWAP
Location	MultiTokenTWAP.sol:69-106, 203-206
Category	Centralisation / Access Control
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-269 (Improper Privilege Management)
Exploit Preconditions	Privileged role registers an illiquid or attacker-controlled pair
Business Impact	A manipulable price could be fed into the rest of the system
Remediation Reference	Open - governance control recommendation

Description

The privileged role can register the pair used for each token price, change the averaging period within bounds and change the liquidity threshold. These controls are necessary for operation but concentrate trust: registering an illiquid or attacker-controlled pair would feed a manipulable price into the rest of the system.

Recommendation

Hold the privileged role in a multi-signature wallet, and review pair registrations, since the role controls which pairs back each token price, the averaging period and the liquidity threshold.

Verification Result: Open. **Residual Risk:** Depends on multi-signature governance and pair-registration review.

F-024: Testnet factory address active in source

Field	Value
ID	F-024
Severity	Informational (deployment-blocking)
Status	Open
Contract	MultiTokenTWAP
Location	MultiTokenTWAP.sol:59-64
Category	Deployment Configuration
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-547 (Use of Hard-coded, Security-relevant Constants)
Exploit Preconditions	Mainnet deployment with the testnet factory still configured
Business Impact	Pair discovery for the spot path would fail or use wrong pairs
Remediation Reference	Open - must be changed before mainnet deployment

Description

The factory address constant is set to the testnet factory while the production address is left commented out. This is expected during testing but must be switched before mainnet deployment, since pair discovery for the spot path depends on this constant.

Recommendation

Before mainnet deployment, switch the factory constant to the production factory address that is currently commented out.

Verification Result: Open. **Residual Risk:** Deployment-blocking; must be resolved before mainnet.

F-025: Deployment dependency order

Field	Value
ID	F-025
Severity	Informational
Status	Open (by design)
Contract	VestingManager
Location	VestingManager.sol:546-555
Category	Deployment Procedure
House Severity	Informational
CVSS v3.1	0.0
CWE ID	N/A (procedural note)
Exploit Preconditions	Deployment without granting the manager role
Business Impact	A window of non-functionality until the role is granted
Remediation Reference	Unchanged by design - procedural note

Description

The manager role is not assigned in the constructor due to circular dependencies with the manager contracts, so it must be granted by a separate call after deployment for the system to be functional.

Recommendation

Ensure the deployment script grants the manager role immediately after deploying the manager contracts to avoid a window of non-functionality.

Client Response

Unchanged by design. The manager role is granted after deployment through a dedicated setter rather than in the constructor, because of the circular dependency. The deployment script must call this setter as part of bring-up.

Verification Result: Open (procedural note, not a code defect). **Residual Risk:** Handled by deployment procedure.

F-026: TVL manipulation via flash loan or donation

Field	Value
ID	F-026
Severity	Informational
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:501-516
Category	Price / Value Manipulation
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-345 (Insufficient Verification of Data Authenticity)
Exploit Preconditions	Attacker inflates contract balance within one transaction
Business Impact	Accelerated releases ahead of schedule
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The release rate previously depended on the contract balance, which can be inflated within a single transaction by a flash loan or a direct token donation. An attacker could spike the perceived value, trigger a maximized release rate and unlock tokens ahead of schedule.

Recommendation

Avoid using the raw contract balance as the metric that drives release acceleration.

Client Response

Addressed. The release rate no longer depends on the contract balance or any TVL measure. Release now follows a deterministic time-and-rate schedule.

Verification Result: Pass. **Residual Risk:** None.

F-027: Reward inflation via price manipulation

Field	Value
ID	F-027
Severity	Informational
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:278-294
Category	Price Oracle Manipulation
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-682 (Incorrect Calculation)
Exploit Preconditions	Attacker crashes the spot price within one transaction
Business Impact	Disproportionate reward minted
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The ecological reward calculation divides a value by the token price, so a near-zero price produces an enormous token reward. Relying on an instantaneous spot price previously let an attacker crash the price within a single transaction and mint a disproportionate reward.

Recommendation

Use a tamper-resistant price for the reward calculation so a single-block price move cannot inflate rewards.

Client Response

Addressed. The ecological reward calculation now reads a cached TWAP through a view function rather than an instantaneous spot price. A conservative per-claim or per-day cap could still be considered as defence in depth.

Verification Result: Pass. **Residual Risk:** None; an optional per-claim cap is suggested as further hardening.

F-028: Incorrect array size in vesting details view function

Field	Value
ID	F-028
Severity	Informational
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:346
Category	Off-by-one / View Function
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-193 (Off-by-one Error)
Exploit Preconditions	None (read-only correctness)
Business Impact	Off-chain interfaces could be confused by an extra empty entry
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

The vesting details view function previously sized its return arrays to eight and iterated one past the defined vesting types, producing an extra zero entry whose length did not match the type definition.

Recommendation

Size the returned arrays to match the number of defined vesting types.

Client Response

Addressed. The view function now initializes arrays with a length of seven and iterates over indices zero through six, matching the seven defined vesting types.

Verification Result: Pass. **Residual Risk:** None.

F-029: Unchecked ERC20 transfer return value

Field	Value
ID	F-029
Severity	Informational
Status	Open
Contract	VestingManager
Location	VestingManager.sol:428
Category	Unchecked Return Value
House Severity	Informational
CVSS v3.1	0.0
CWE ID	CWE-252 (Unchecked Return Value)
Exploit Preconditions	A token that returns false instead of reverting
Business Impact	None with the in-system token, which reverts on failure
Remediation Reference	Open - hardening recommendation

Description

The release payout moves tokens with transfer and ignores the boolean return value. The in-system token reverts on a failed transfer, so funds cannot be silently lost, and a preceding balance check further reduces the practical risk.

Recommendation

Use SafeERC20 safeTransfer for the release payout, or continue relying on the in-system token reverting on failure.

Verification Result: Open. **Residual Risk:** Informational.

F-030: Gas cost from multiple transfer events per transaction

Field	Value
ID	F-030
Severity	Gas Optimization
Status	Open
Contract	HeartToken
Location	HeartToken.sol:155-177
Category	Gas Optimization
House Severity	Gas Optimization
CVSS v3.1	0.0
CWE ID	N/A (efficiency)
Exploit Preconditions	None
Business Impact	Higher gas cost for taxed trades; micro-transactions less efficient
Remediation Reference	Open - deliberate design trade-off

Description

Each taxable trade executes several separate transfers, one for the recipient and one for each fee destination (burn address, reserve wallet, market wallet). This increases gas cost relative to a plain token transfer, though it is a necessary consequence of the fee-on-transfer design.

Recommendation

Consider distributing fees only when they accumulate to a threshold to reduce the per-transaction gas burden, while accepting that this adds complexity to the contract logic.

Client Response

Unchanged. A taxed trade still performs separate transfers for the recipient, the burn address, the reserve wallet and the market wallet. This is an inherent cost of the fee-on-transfer design and a deliberate trade-off rather than a defect.

Verification Result: Open (accepted trade-off). **Residual Risk:** None (efficiency only).

F-031: Gas-limit denial of service via unbounded array iteration

Field	Value
ID	F-031
Severity	Gas Optimization (potential DoS)
Status	Verified
Contract	VestingManager
Location	VestingManager.sol:189-190
Category	Resource Consumption / DoS
House Severity	Gas Optimization
CVSS v3.1	0.0
CWE ID	CWE-834 (Excessive Iteration)
Exploit Preconditions	A user accumulates a very large number of schedules
Business Impact	Iteration could exceed the block gas limit, locking claims/withdrawals
Remediation Reference	Verified against remediated snapshot (re-audit, 2026-06-12)

Description

Several functions iterate over the full array of a user's schedules in a single transaction. Without a limit on the number of schedules a user could create, the iteration cost could grow until it exceeds the block gas limit, permanently preventing the user from claiming rewards or withdrawing principal.

Recommendation

Limit the maximum number of active schedules per user so iteration cannot exceed the block gas limit.

Client Response

Addressed. The number of schedules per user and type is now capped, and schedule creation reverts once the cap is reached, bounding the length of the iterated arrays.

Verification Result: Pass - the denial-of-service path is removed. **Residual Risk:** Iteration cost remains proportional to the number of schedules, so the cap should be kept conservative.

Centralisation, Access Control & Privilege Analysis

Overview

The HeartLive system concentrates configuration and pricing authority in a small number of privileged roles and a single off-chain backend signer. The most serious on-chain manipulation paths were closed during remediation (pricing now flows through the TWAP oracle, signatures are chain-bound), so the residual centralisation risk is primarily governance-based: privileged changes take effect immediately, and trust rests on the roles being held by a multi-signature wallet. The token itself cannot mint new supply and caps its tax at one percent, which materially limits owner power over holders.

On-Chain Roles & Permissions

Role / Address	Permissions	Timelock	Multisig	Risk Level
DAO / Governance (DAO_ROLE)	Set bounded parameters, wallets, point types, backend signer; grant/revoke roles	No	Intended (Gnosis Safe)	Medium
Admin (ADMIN_ROLE)	Administrative configuration in the parameters registry	No	Intended	Medium
Token Owner (HeartToken)	Set taxed pairs, reserve/market wallets, grant manager/external roles; cannot mint; tax capped at 1%	No	Intended	Medium
Manager / External role (VestingManager)	Create vesting schedules and process claims on behalf of users	No	Intended	Medium
Oracle Governance (MultiTokenTWAP)	Register pairs, set averaging period (bounded) and liquidity threshold	No	Intended	Medium

Common protective properties observed: the last holder of a governance role cannot be removed (preventing an unmanageable system); wallet and signer setters reject the zero address and no-op updates; and parameters are bounded by per-key minimum and maximum values.

Centralisation Risk Summary

Risk	Severity	Recommendation
Privileged roles take effect immediately (no timelock)	Medium (F-008)	Hold roles in a multi-signature wallet and add an on-chain timelock
Backend signer is a single trusted off-chain key	High if compromised	Store in KMS/HSM, enforce rotation, monitor for anomalous signing
Oracle governance can register the pair backing each price	Informational (F-023)	Multi-signature control and mandatory review of pair registrations
Manager role can act on behalf of users	Medium	Multi-signature control and monitoring of schedule/claim activity

Conclusion

Summary

The HeartLive smart contract system was reviewed across seven contracts covering the token, a central parameter registry, a TWAP price oracle, a vesting and reward engine, and three manager contracts. The initial review identified three High severity issues - a vesting insolvency caused by a missing token transfer, an unbounded promotion withdrawal, and single-block price manipulation through spot reserves - together with several Medium and Low severity issues. All Critical and High severity findings, and the majority of Medium and Low findings, were remediated and independently verified. Pricing across the system now depends on a tamper-resistant TWAP read through a cached view, and backend-signed authorization is bound to the chain and the contract instance.

The remaining risk is concentrated in two areas: centralisation of privileged control (immediate-effect roles and a trusted off-chain signer, to be mitigated by multi-signature governance and a timelock), and two deployment configuration constants that still point at test values. Neither area represents an open Critical or High severity vulnerability, but the deployment constants are strictly blocking for a mainnet release.

Positive Observations

- Serious issues were fixed thoroughly and at the root cause rather than patched superficially; the pricing redesign around a shared TWAP oracle is a strong architectural improvement.
- The token has a fixed, capped supply minted once at construction, no open mint path, and a hard one-percent ceiling on the combined transfer tax.
- Defensive engineering is consistent: reentrancy guards on value-moving functions, per-recipient balance checks, bounded parameters, last-admin protection, and (after remediation) broad event coverage for off-chain monitoring.
- Signature flows include replay protection through a per-user ring buffer in addition to chain and contract binding.

Critical Action Items

1. Switch the mock USDT constant (DAOParameters) and the testnet PancakeSwap factory constant (MultiTokenTWAP) to their production values before mainnet deployment (F-020, F-024).
2. Register every priced token in the TWAP oracle before it is used, and consider reverting on the spot-price fallback for unregistered tokens (F-015).
3. Hold all privileged roles (DAO, admin, token owner, manager, oracle governance) in a multi-signature wallet, and place critical configuration changes behind an on-chain timelock (F-008, F-023).
4. Protect and monitor the backend signer key (KMS/HSM, rotation, anomaly alerting), since it authorizes purchases and exchanges.

Glossary

Abbreviation	Definition
ABI	Application Binary Interface
AMM	Automated Market Maker
BSC	BNB Smart Chain
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
dApp	Decentralised Application
DeFi	Decentralised Finance
DoS	Denial of Service
EOA	Externally Owned Account
ERC20	Ethereum token standard (BEP-20 on BSC)
EVM	Ethereum Virtual Machine
HSM	Hardware Security Module
HTL	Heartlive Token
KMS	Key Management Service
LoC	Lines of Code
MFA	Multi-Factor Authentication
NatSpec	Ethereum Natural Language Specification Format
PoC	Proof of Concept
RBAC	Role-Based Access Control
SAST	Static Application Security Testing
TVL	Total Value Locked
TWAP	Time-Weighted Average Price



Solidproof.io - Trust Made in Germany

Blockchain Security | Smart Contract Audits | KYC | Marketing

FutureVisions Deutschland UG (haftungsbeschränkt)

Heideland-Ost 34
24976 Handewitt
Germany, Deutschland

Sitz der Gesellschaft / Amtsgericht: Flensburg
HRB 16612 FL
USt-IdNr. DE364783510
Geschäftsführer: Mails Nielsen, Kevin Arens

Solidproof.io is a product of
FutureVisions Deutschland

Please note: You automatically accept our Terms and Conditions and Privacy Policy when using our services.

You can find the current version on our website: <https://solidproof.io>.



**Blockchain Security | Smart Contract Audits | KYC
Development | Marketing**


MADE IN GERMANY